Homework 3

Name: Hao Pang
Course: DSCI 552 — Machine Learning for Data Science

1. Time Series Classification Part 1: Feature Creation/Extraction

(a) Load and Inspect the Data

The AReM dataset contains 88 instances from seven activity folders. Each instance includes six sensor time series. First, I check the data folder and count the CSV files.

In [1]:
```
from pathlib import Path

# Location of the data folder
data_path = Path("AReM")

print("Data folder exists:", data_path.exists())

# Find all CSV files
csv_files = list(data_path.glob("*/dataset*.csv"))

print("Number of CSV files:", len(csv_files))
```

```
Data folder exists: True
Number of CSV files: 88
```

I inspect one file to check its columns and number of observations. The first column contains time, and the remaining six columns contain sensor measurements.

In [2]:
```
import pandas as pd

# Column names: time and six sensor series
column_names = [
    "time",
    "avg_rss12", "var_rss12",
    "avg_rss13", "var_rss13",
    "avg_rss23", "var_rss23"
]

# Read one example file
sample_file = data_path / "bending1" / "dataset1.csv"

sample_df = pd.read_csv(
    sample_file,
    comment="#",
    header=None,
    names=column_names
)

print("Data shape:", sample_df.shape)
```

```
sample_df.head()
```

```
Data shape: (480, 7)
```

Out[2]:

	time	avg_rss12	var_rss12	avg_rss13	var_rss13	avg_rss23	var_rss23
0	0	39.25	0.43	22.75	0.43	33.75	1.3
1	250	39.25	0.43	23.00	0.00	33.00	0.0
2	500	39.25	0.43	23.25	0.43	33.00	0.0
3	750	39.50	0.50	23.00	0.71	33.00	0.0
4	1000	39.50	0.50	24.00	0.00	33.00	0.0

Read and Check All Data Files

Some files use spaces instead of commas, and some have an extra comma at the end of a line. I handle these formatting differences when reading the files, without changing the original data.

I also check the number of observations and missing cells in each file.

In [3]:

```
# Read one file and handle different separators
def read_activity_file(file_path):
    rows = []

    with open(file_path, "r") as file:
        for line in file:
            line = line.strip()

            # Skip comments and empty lines
            if not line or line.startswith("#"):
                continue

            # Handle commas, spaces, and trailing commas
            values = line.replace(",", " ").split()

            if len(values) != 7:
                raise ValueError(f"Unexpected number of columns: {file_path}")

            rows.append([float(value) for value in values])

    return pd.DataFrame(rows, columns=column_names)


# Store each file separately
data = {}
check_results = []

for file_path in sorted(csv_files):
    activity = file_path.parent.name
    file_name = file_path.name
    key = f"{activity}/{file_name}"

    df = read_activity_file(file_path)
    data[key] = df
```

```
    check_results.append({
        "activity": activity,
        "file": file_name,
        "rows": len(df),
        "missing_cells": int(df.isna().sum().sum())
    })

check_df = pd.DataFrame(check_results)

print("Files loaded:", len(data))
print("Total missing cells:", check_df["missing_cells"].sum())

print("\nNumber of files by row count:")
print(check_df["rows"].value_counts().sort_index())

print("\nFiles with an unexpected row count:")
check_df[check_df["rows"] != 480]
```

```
Files loaded: 88
Total missing cells: 0

Number of files by row count:
rows
479     1
480    87
Name: count, dtype: int64

Files with an unexpected row count:
```

Out[3]:

	activity	file	rows	missing_cells
56	sitting	dataset8.csv	479	0

(b) Split the Data into Training and Test Sets

For bending1 and bending2, I use dataset1 and dataset2 as test data. For the other activities, I use dataset1, dataset2, and dataset3 as test data. All remaining files are used as training data.

In [4]:

```
train_data = {}
test_data = {}
split_records = []

for key, df in data.items():
    # Example key: bending1/dataset1.csv
    activity, file_name = key.split("/")

    # Extract the dataset number from the filename
    dataset_number = int(
        file_name.replace("dataset", "").replace(".csv", "")
    )

    # Apply the required test-set rule
    if activity in ["bending1", "bending2"]:
        is_test = dataset_number in [1, 2]
    else:
        is_test = dataset_number in [1, 2, 3]
```

```
    if is_test:
        test_data[key] = df
        split = "test"
    else:
        train_data[key] = df
        split = "train"

    split_records.append({
        "activity": activity,
        "file": file_name,
        "split": split
    })

split_df = pd.DataFrame(split_records)

print("Training instances:", len(train_data))
print("Test instances:", len(test_data))

# Show the number of instances in each activity
split_summary = pd.crosstab(
    split_df["activity"],
    split_df["split"]
)

split_summary = split_summary[["train", "test"]]
split_summary["total"] = split_summary.sum(axis=1)

split_summary
```

```
Training instances: 69
Test instances: 19
```

Out[4]:

split	train	test	total
activity			
bending1	5	2	7
bending2	4	2	6
cycling	12	3	15
lying	12	3	15
sitting	12	3	15
standing	12	3	15
walking	12	3	15

(c) Feature Extraction

(i) Common Time-Domain Features

Time-domain features summarize the values and patterns in a time series. They can describe its typical level, variation, distribution shape, and how values change over time. Common examples are listed below.

Category	Feature	Description
Level	Minimum	The smallest value.
Level	Maximum	The largest value.
Level	Mean	The average value.
Level	Median	The middle value after sorting.
Level	Root mean square (RMS)	The square root of the average of the squared values.
Spread	Standard deviation	How much the values vary around the mean.
Spread	Variance	The square of the standard deviation.
Spread	Range	The difference between the maximum and minimum.
Spread	First / third quartile (Q1, Q3)	The 25th and 75th percentiles.
Spread	Interquartile range (IQR)	Q3 − Q1; a spread measure robust to outliers.
Shape	Skewness	How asymmetric the distribution is.
Shape	Kurtosis	How heavy the tails of the distribution are.
Temporal	Autocorrelation	How strongly values are related to earlier values at a given lag.
Temporal	Mean crossing rate	How often the series crosses its mean.
Temporal	Linear trend (slope)	The overall upward or downward trend.
Temporal	Number of peaks	The number of local maxima.
Temporal	Absolute sum of changes	The sum of absolute differences between consecutive values.

For this assignment, I extract minimum, maximum, mean, median, standard deviation, Q1, and Q3 from each of the six sensor time series.

References:

- Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time Series FeatuRe Extraction on basis of Scalable Hypothesis tests (tsfresh – A Python package). *Neurocomputing*, 307, 72–77.
- tsfresh documentation: Feature calculators

(ii) Extract Time-Domain Features

I calculate seven features for each of the six sensor time series. This gives 42 features for each instance. The time column is not included.

I keep the features on their original scales and use the sample standard deviation (ddof=1). For sitting/dataset8.csv, I use its 479 available rows.

In [5]:
```
# Extract 42 features from one activity instance
def extract_features(df):
    features = {}
```

```
    # Skip the time column
    for i, column in enumerate(column_names[1:], start=1):
        series = df[column]

        features[f"min{i}"] = series.min()
        features[f"max{i}"] = series.max()
        features[f"mean{i}"] = series.mean()
        features[f"median{i}"] = series.median()
        features[f"std{i}"] = series.std(ddof=1)
        features[f"q1_{i}"] = series.quantile(0.25)
        features[f"q3_{i}"] = series.quantile(0.75)

    return features


# Extract features from all 88 instances
feature_rows = {}

for key, df in data.items():
    feature_rows[key] = extract_features(df)

features_df = pd.DataFrame.from_dict(
    feature_rows,
    orient="index"
)

features_df.index.name = "instance"

print("Feature matrix shape:", features_df.shape)
print("Missing feature values:", features_df.isna().sum().sum())

features_df.head()
```

```
Feature matrix shape: (88, 42)
Missing feature values: 0
```

Out[5]:

instance	min1	max1	mean1	median1	std1	q1_1	q3_1	min2
bending1/dataset1.csv	37.25	45.00	40.624792	40.50	1.476967	39.25	42.00	0.0
bending1/dataset2.csv	38.00	45.67	42.812812	42.50	1.435550	42.00	43.67	0.0
bending1/dataset3.csv	35.00	47.40	43.954500	44.33	1.558835	43.00	45.00	0.0
bending1/dataset4.csv	33.00	47.75	42.179812	43.50	3.670666	39.15	45.00	0.0
bending1/dataset5.csv	33.00	45.75	41.678063	41.75	2.243490	41.33	42.75	0.0

5 rows × 42 columns

In [6]:

```
# Keep the same train/test split
train_features = features_df.loc[list(train_data.keys())].copy()
test_features = features_df.loc[list(test_data.keys())].copy()

print("Training features:", train_features.shape)
print("Test features:", test_features.shape)
```

```
Training features: (69, 42)
Test features: (19, 42)
```

(iii) Standard Deviations and Bootstrap Confidence Intervals

I calculate the standard deviation of each feature across the 88 instances.

Then I use bootstrap sampling to find a 90% confidence interval for each standard deviation. I sample 88 instances with replacement and repeat this 10,000 times. The 5th and 95th percentiles give the interval limits.

I use a random seed of 42 so that the results can be reproduced.

In [7]:
```python
import numpy as np

# Standard deviation of each feature across all 88 instances
feature_std = features_df.std(ddof=1)

# Convert the feature table to a NumPy array
values = features_df.to_numpy()
n_instances, n_features = values.shape

# Bootstrap settings
n_bootstrap = 10000
rng = np.random.default_rng(42)

bootstrap_std = np.empty((n_bootstrap, n_features))

for b in range(n_bootstrap):
    # Sample 88 complete rows with replacement
    indices = rng.choice(
        n_instances,
        size=n_instances,
        replace=True
    )

    sample = values[indices]

    # Calculate the standard deviation of each column
    bootstrap_std[b] = sample.std(axis=0, ddof=1)

# Limits of the 90% percentile bootstrap intervals
lower = np.percentile(bootstrap_std, 5, axis=0)
upper = np.percentile(bootstrap_std, 95, axis=0)

# Put the results into one table
bootstrap_results = pd.DataFrame({
    "std_estimate": feature_std,
    "CI_90_lower": lower,
    "CI_90_upper": upper
})

bootstrap_results.index.name = "feature"

print("Result shape:", bootstrap_results.shape)

bootstrap_results.round(4)
```

```
Result shape: (42, 3)
```

Out[7]:

feature	std_estimate	CI_90_lower	CI_90_upper
min1	9.5700	8.2591	10.7762
max1	4.3944	3.3243	5.2928
mean1	5.3357	4.6952	5.8836
median1	5.4401	4.7909	6.0076
std1	1.7722	1.5701	1.9549
q1_1	6.1536	5.5646	6.6560
q3_1	5.1389	4.3111	5.8451
min2	0.0000	0.0000	0.0000
max2	5.0627	4.6254	5.4020
mean2	1.5742	1.3977	1.7068
median2	1.4122	1.2372	1.5456
std2	0.8841	0.8016	0.9414
q1_2	0.9464	0.8299	1.0351
q3_2	2.1253	1.8952	2.2955
min3	2.9565	2.7542	3.1094
max3	4.8751	4.1600	5.4677
mean3	4.0084	3.4139	4.4885
median3	4.0364	3.4230	4.5353
std3	0.9467	0.7626	1.1228
q1_3	4.2207	3.6227	4.6934
q3_3	4.1716	3.5336	4.6945
min4	0.0000	0.0000	0.0000
max4	2.1836	1.9737	2.3590
mean4	1.1661	1.0749	1.2220
median4	1.1456	1.0535	1.2008
std4	0.4582	0.4208	0.4857
q1_4	0.8436	0.7740	0.8910
q3_4	1.5525	1.4308	1.6268
min5	6.1240	4.4404	7.4967
max5	5.7412	4.7288	6.5651
mean5	5.6756	4.4239	6.7119
median5	5.8138	4.5210	6.8826

feature	std_estimate	CI_90_lower	CI_90_upper
std5	1.0249	0.8141	1.2172
q1_5	6.0965	4.8040	7.1827
q3_5	5.5317	4.3563	6.5198
min6	0.0458	0.0000	0.0785
max6	2.5189	2.2492	2.7679
mean6	1.1548	1.0624	1.2150
median6	1.0865	0.9942	1.1477
std6	0.5176	0.4788	0.5449
q1_6	0.7586	0.6890	0.8083
q3_6	1.5236	1.4018	1.6010

Interpretation of the bootstrap results:

- **Constant features:** min2 and min4 have a standard deviation of 0 and a confidence interval of [0, 0]. The minimum of these variance series is 0 in every instance, so these two features carry no information. min6 is almost constant as well (std ≈ 0.05, CI [0, 0.08]).
- **Effect of scale:** Features from the average series (1, 3, 5) have much larger standard deviations (e.g., min1 ≈ 9.57, q1_1 ≈ 6.15, min5 ≈ 6.12) than features from the variance series (2, 4, 6), which are mostly below 2.5. This is mainly because the average series take larger values (roughly 10–50) than the variance series (roughly 0–5). Raw standard deviations are therefore not directly comparable across features. Standardizing the features, or using the coefficient of variation, would allow a fairer comparison.
- **Uncertainty:** Apart from the constant minimum features, every interval lies well above 0, so these features do vary across the 88 instances. Some intervals are wide relative to their estimates (e.g., min5: [4.44, 7.50], mean5: [4.42, 6.71]), which means that with only 88 instances these standard deviations are estimated with considerable uncertainty.

(iv) Select Three Features

To support my choice with data, I use two checks on the **training set only**, so the test set is not used for feature selection:

1. **ANOVA F-statistic:** how well each feature separates the seven activities. A larger F means the activity means differ more relative to the variation within each activity. I take the median F across the six time series for each feature type.
2. **Correlation between feature types:** two features that are highly correlated carry mostly the same information, so selecting both adds little.

In [8]:

```python
from sklearn.feature_selection import f_classif

# Use the training set only, so the test set is not used for selection
X_train = train_features
y_train = [key.split("/")[0] for key in X_train.index]

# Remove features that are constant in the training set (F is undefined)
non_constant = X_train.columns[X_train.std() > 0]
print("Constant features removed:", list(X_train.columns.difference(non_constant

# One-way ANOVA F-statistic: how well each feature separates the 7 activities
F_values, p_values = f_classif(X_train[non_constant], y_train)

f_table = pd.DataFrame({
    "feature": non_constant,
    "F": F_values
})

# Feature type, e.g. "mean3" -> "mean", "q1_3" -> "q1"
f_table["type"] = f_table["feature"].str.replace(r"_?\d$", "", regex=True)

# Median F across the six time series for each feature type
f_by_type = (
    f_table.groupby("type")["F"]
    .median()
    .sort_values(ascending=False)
    .round(1)
)

print("\nMedian F-statistic by feature type (training set):")
print(f_by_type)

# Check redundancy: average correlation between feature types
# across the six time series
feature_types = ["min", "max", "mean", "median", "std", "q1", "q3"]
corr_list = []

for i in range(1, 7):
    cols = [f"{t}_{i}" if t.startswith("q") else f"{t}{i}"
            for t in feature_types]
    corr_list.append(X_train[cols].corr().to_numpy())

# Ignore NaN from constant features (min2, min4) when averaging
avg_corr = pd.DataFrame(
    np.nanmean(corr_list, axis=0),
    index=feature_types,
    columns=feature_types
)

print("\nAverage correlation between feature types (training set):")
avg_corr.round(2)
```

```
Constant features removed: ['min2', 'min4']

Median F-statistic by feature type (training set):
type
mean      307.4
q3        297.1
median    252.8
q1        135.2
std        71.9
max        32.3
min        16.8
Name: F, dtype: float64

Average correlation between feature types (training set):
```

Out[8]:

	min	max	mean	median	std	q1	q3
min	1.00	0.36	0.59	0.57	-0.32	0.60	0.52
max	0.36	1.00	0.79	0.75	0.63	0.71	0.82
mean	0.59	0.79	1.00	0.99	0.38	0.98	0.98
median	0.57	0.75	0.99	1.00	0.36	0.98	0.96
std	-0.32	0.63	0.38	0.36	1.00	0.28	0.50
q1	0.60	0.71	0.98	0.98	0.28	1.00	0.93
q3	0.52	0.82	0.98	0.96	0.50	0.93	1.00

Results and choice: I choose **mean, standard deviation, and maximum**.

- **Mean** has the highest median F (about 307), so it separates the activities best. Median, Q1, and Q3 also score high, but they are almost perfectly correlated with the mean (r ≈ 0.93–0.99), so they add little new information. I keep only the mean from this group.
- **Standard deviation** has a moderate F (about 72) and a low correlation with the mean (r ≈ 0.38). It captures how much the signal fluctuates, which is different information from the signal level.
- **Maximum** has a higher F than minimum (about 32 vs. 17). Minimum is weak partly because the variance series (2, 4, 6) almost always reach 0, and min2 and min4 are constant across all instances. Maximum is moderately correlated with the mean (r ≈ 0.79), but it adds information about peak values.

Limitation: The F-statistic looks at one feature at a time and does not show how features work together in a model. Maximum can also be affected by outliers. I will check this choice with classification models in Homework 4.

In [9]:
```python
# Select three feature types from each of the six time series
selected_columns = []

for i in range(1, 7):
    selected_columns.extend([
        f"mean{i}",
        f"std{i}",
        f"max{i}"
```

```
    ])

selected_features = features_df[selected_columns].copy()

print("Selected feature matrix:", selected_features.shape)

selected_features.head()
```

```
Selected feature matrix: (88, 18)
```

Out[9]:

feature instance	mean1	std1	max1	mean2	std2	max2	mean3
bending1/dataset1.csv	40.624792	1.476967	45.00	0.358604	0.322605	1.30	19.040937
bending1/dataset2.csv	42.812812	1.435550	45.67	0.372437	0.289158	1.22	20.096979
bending1/dataset3.csv	43.954500	1.558835	47.40	0.426250	0.338690	1.70	22.122354
bending1/dataset4.csv	42.179812	3.670666	47.75	0.696042	0.630860	3.00	22.183625
bending1/dataset5.csv	41.678063	2.243490	45.75	0.535979	0.405469	2.83	19.006563

2. ISLR 3.7.4

(a)

The cubic model has a lower or equal training RSS. It includes the linear model as a special case: when the coefficients of X^2 and X^3 are zero, it becomes a linear model. Therefore, least squares can fit the training data at least as well.

(b)

I would expect the linear model to have a lower test RSS. The true relationship is linear, so the extra terms are not needed. The cubic model may fit noise in the training data and perform worse on new data. This is an expected result, not a guarantee for every test set.

(c)

The cubic model still has a lower or equal training RSS. The reason is the same as in part (a): it includes the linear model as a special case. This does not depend on the true relationship.

(d)

There is not enough information to decide. The cubic model may have a lower test RSS if its extra terms help describe the true relationship. However, if the relationship is close to linear, the extra terms may mainly fit noise. The result depends on whether the better fit is worth the added variance.